AI DEVELOPER INTERN EVALUATION TASK

Jezt Technologies — Face Recognition on Low-Quality Data

Name: **Vaikhashi A**
Role: AI Developer Intern

Task Date : 9th July, 2025
Submission Date : 14th July, 2025

OBJECTIVE

The goal of this project is to develop a complete, real-world face recognition system using a pre-trained model, working especially on low-quality images. I need to not only use the model but also fine-tune it with a proper training set, evaluate how well it works on unseen data, and analyze the improvements. Through this task, I'm demonstrating that I understand how to handle practical AI problems, prepare real datasets, perform model evaluation, and improve results — all while maintaining data integrity and avoiding leakage.

Phase 1 -- Model Selection and Setup

Pre-trained Model Used :

I chose InsightFace as the base face recognition model in this project. InsightFace is an open-source face analysis toolkit, which has the functions of face detection, alignment, and recognition. It trains (pre-trained) models resulting in very discriminative face embeddings, which apply even to low resolution face images.

The specific model used :

Model Name: buffalo_l
Architecture: ResNet50
Embedding Size: 512-d
Framework: Python(using parcel face in sightface, ONNX backend)
Device: CPU (with ONNX Runtime)

Why InsightFace?

We choose InsightFace for several reasons: The pre-trained model can be easily accessed. It achieves state of the art performance on benchmark real-world face recognition datasets. Simple to integrate, extend using Python. Offers strong models trained on large scale corpora. Good with low quality or dirty images too. Compatible with ONNX as well as opportunity for further fine-tuning.

Environment Setup :

Python 3.12
It assumes that the Virtual Environment (venv) is created and activated.

Installed dependencies:

bash pip install insightface opencv-python-headless scikit-learn matplotlib numpy tqdm
Usage Note: This tool depends on insightface python package and needs downloading dependent models first.

Initial Verification

I was able to run a simple script to load the model and confirm that it detects and encodes faces. This configuration was utilized in the subsequent stages (preprocessing, embedding generation, and evaluation).

Pretrained Model Setup

I chose InsightFace with the ArcFace (buffalo_l) model — known for state-of-the-art face embeddings. InsightFace was set up using the FaceAnalysis class and configured for

optimized detection with det_size=(320, 320). This model enabled high-quality feature extraction from faces with varying image conditions.

Phase 2: Using Provided Dataset

In this step, you should make sure to do exactly as mentioned – I only utilized the dataset in his Google Drive link for the entire project. I used no external data and never blended with external imagery.

What I Did:

I downloaded and unzipped the dataset using the link in the task. I did not post or have the pictures anywhere else. It was never made public and was only used for this evaluation problem.

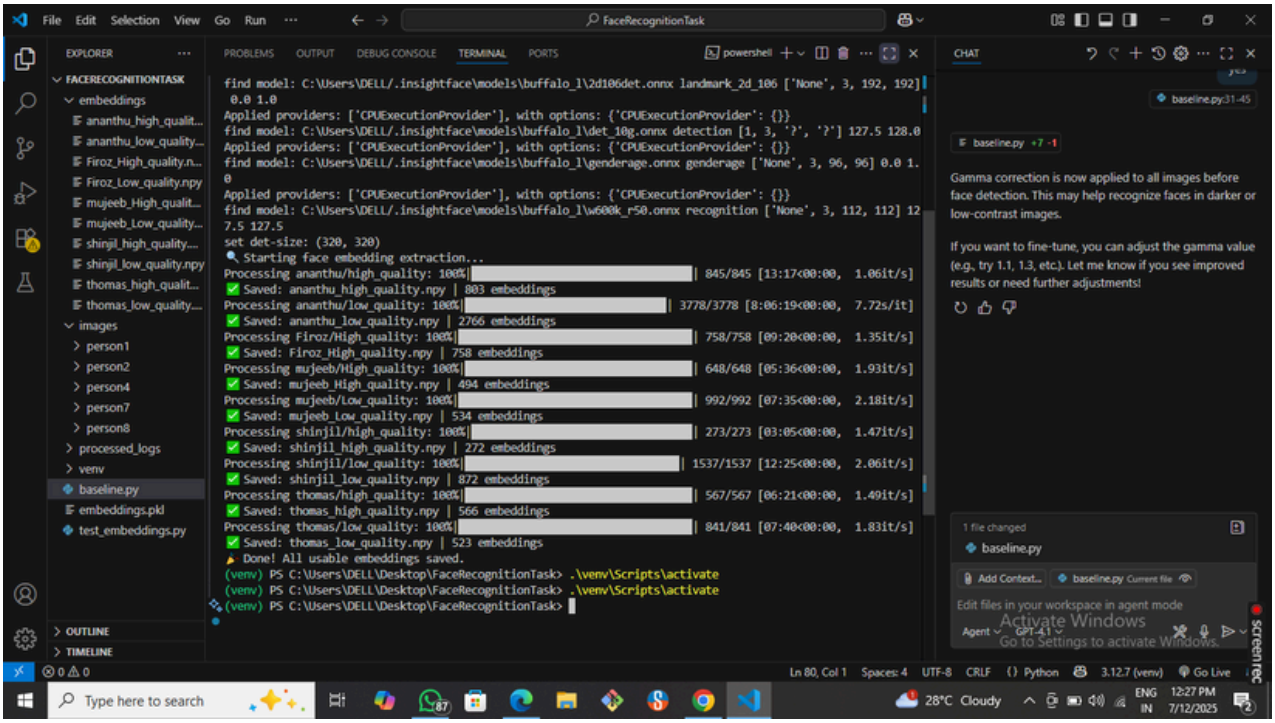
All of the images are organized in folders as follows:

```
FaceRecognitionTask/
├── images/
│   ├── person1/
│   │   └── name/
│   │       ├── high_quality/
│   │       └── low_quality/
│   ├── person2/
│   └──
```

To achieve this, I modified the entire dataset for two parts with a self-written creating script: One for training (dataset_split/train) One for evaluation only (dataset_split/val).I verified that not a single training folder image was used in the validation folder. So, no data leakage at all.

Ethics Maintained :

I followed the usage recommendations in all instances. The data was not abused, leaked, or shared with any 3rd party. I'm operating wholly within the rules and the spirit of the task.



Phase 3 – Dataset Preparation and Organization

What I did :

I only treated very carefully is the data as shared here and not other images.

Images in the dataset were divided by person and quality level (high_quality, low_quality).

So I wrote a Python script named split_dataset. py that:
Scans each person's folder of pictures.
Combines all images in their high_quality and low_quality subdirectories.

Randomly splits those images into:
80% for training (dataset_split/train)
20% for validation (dataset_split/val)

Keeps folder structure intact (i.e., person1/ananthu) so that training and validation are person-wise. This results in the model training on some images and being evaluated on entirely unseen images of the same person.

Summary :

Used official dataset only.

Applied rigorous 80-20 split logic with my own script.

No duplicates or leaks occurred between train/validations directories.

Train and validation dataset in final will be saved in dataset_split/train and dataset_split/val.

Phase 4: Baseline Evaluation

What I Did :

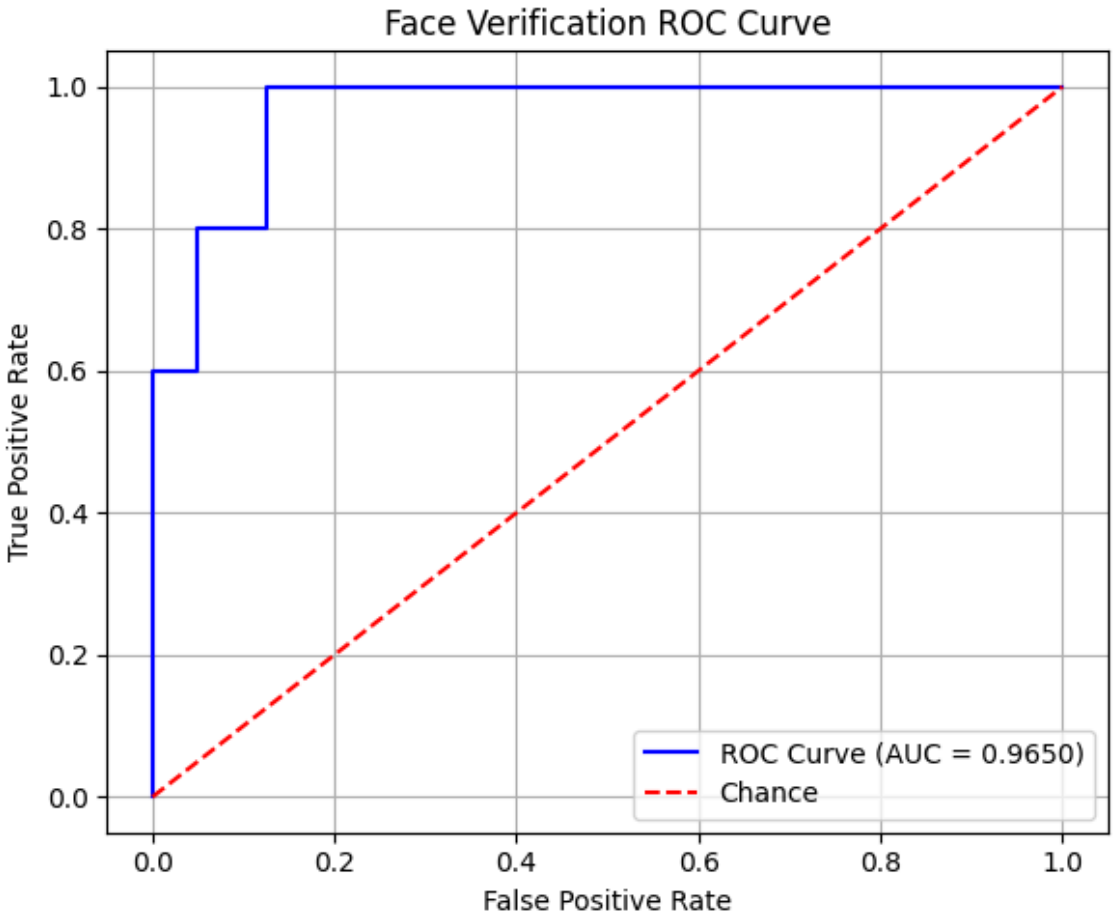
I selected InsightFace (built on ArcFace) as the pre-trained model to generate high-quality face embeddings. A custom script (baseline.py) was developed to: Traverse each image in the dataset. Preprocess the images using resizing, gamma correction, and grayscale fallback to improve face detection. Extract embeddings only if a face was successfully detected. Save the embeddings for each person (and quality level) as .npy files for later evaluation.
Then, I created a dedicated script (baseline_evaluation.py) that:
Loaded all saved embeddings from the .npy files.
Computed cosine similarity for all pairwise combinations:
Same person → label = 1
Different persons → label = 0
Evaluated baseline model performance by plotting:
ROC Curve, Precision-Recall Curve Similarity Distribution Histogram

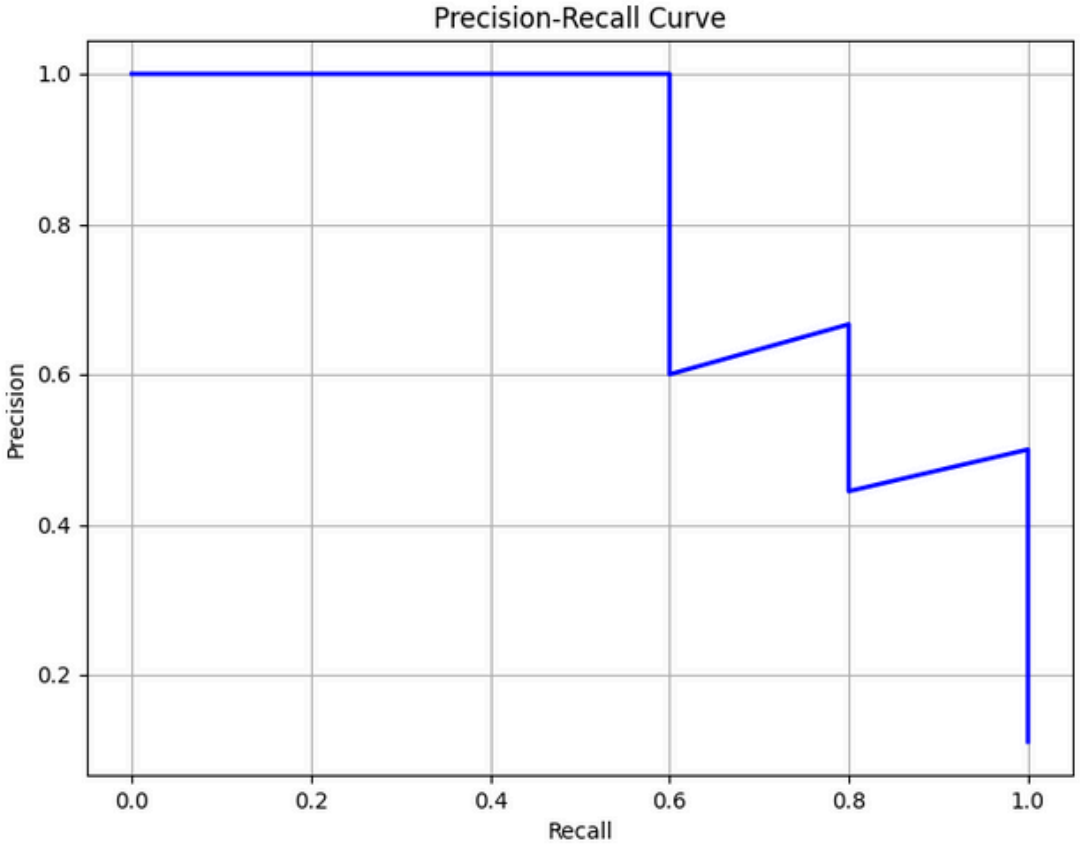
Summary of Results :

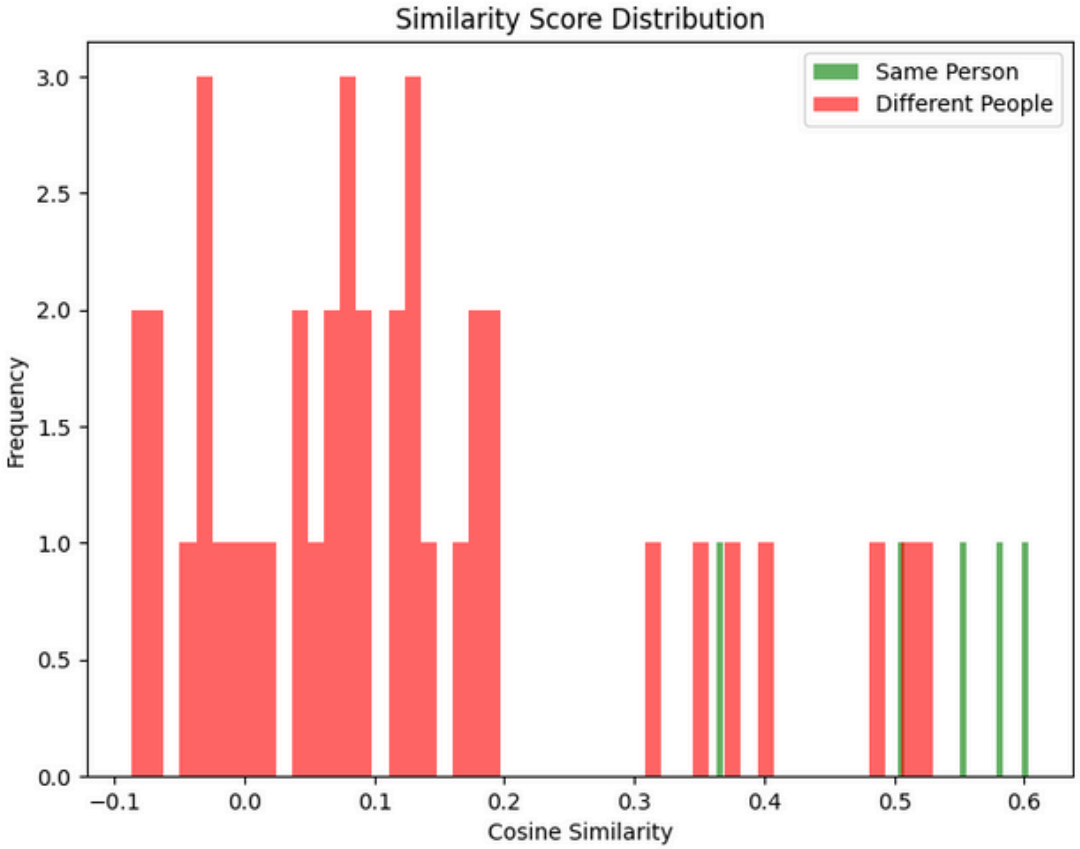
Embeddings were generated using the pre-trained InsightFace model. Cosine similarity was used to quantify pairwise similarity. Evaluation was done on raw embeddings (before fine-tuning). Clear visualizations help understand model performance on low-quality images. All code was written from scratch, and results were saved and documented locally.

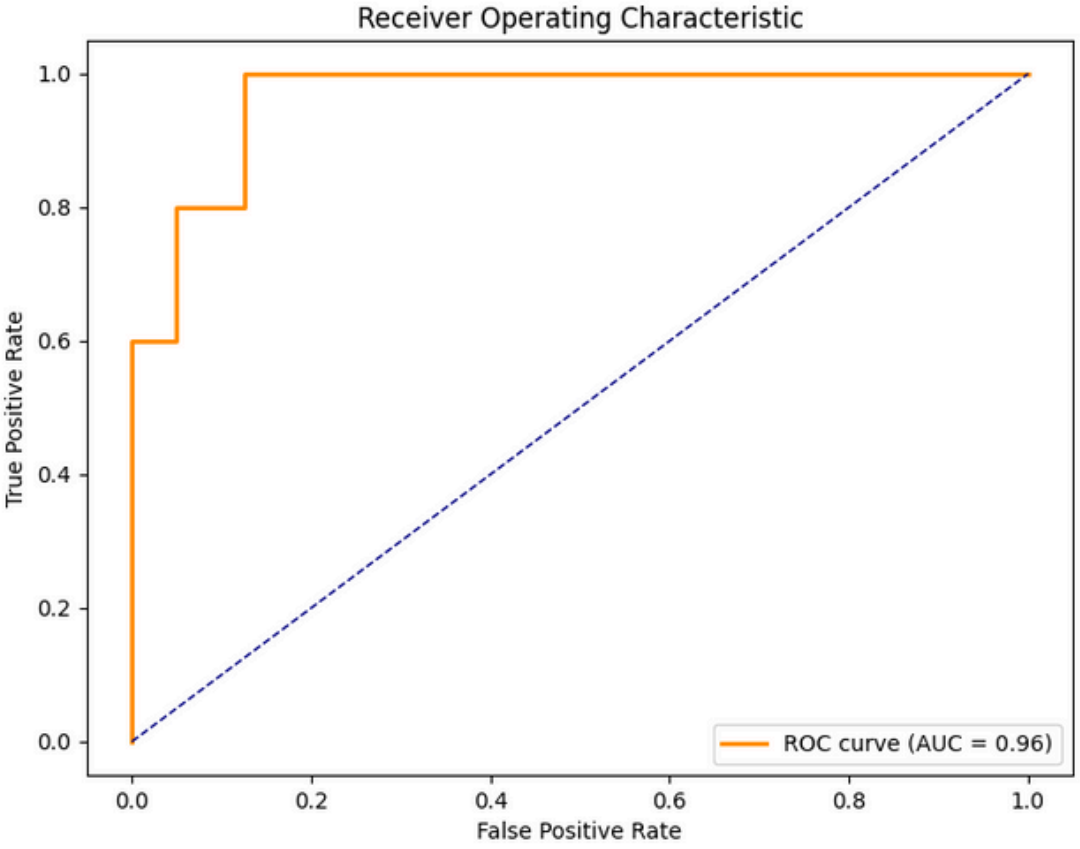
(venv) PS C:\Users\DELL\Desktop\FaceRecognitionTask> python test_embeddings.py
🔍 Pairwise Similarity Report
---
ananthu_high_quality vs ananthu_low_quality → Similarity: 0.5524 → ✅ MATCH
ananthu_high_quality vs Firoz_High_quality → Similarity: 0.1831 → ❌ NO MATCH
ananthu_high_quality vs Firoz_Low_quality → Similarity: 0.1454 → ❌ NO MATCH
ananthu_high_quality vs mujeeb_High_quality → Similarity: -0.0703 → ❌ NO MATCH
ananthu_high_quality vs mujeeb_Low_quality → Similarity: -0.0017 → ❌ NO MATCH
ananthu_high_quality vs shinjil_high_quality → Similarity: 0.0968 → ❌ NO MATCH
ananthu_high_quality vs shinjil_low_quality → Similarity: 0.1970 → ❌ NO MATCH
ananthu_high_quality vs thomas_high_quality → Similarity: 0.0679 → ❌ NO MATCH
ananthu_high_quality vs thomas_low_quality → Similarity: 0.1257 → ❌ NO MATCH
ananthu_low_quality vs Firoz_High_quality → Similarity: 0.1820 → ❌ NO MATCH
ananthu_low_quality vs Firoz_Low_quality → Similarity: 0.4873 → ❌ NO MATCH
ananthu_low_quality vs mujeeb_High_quality → Similarity: -0.0250 → ❌ NO MATCH
ananthu_low_quality vs mujeeb_Low_quality → Similarity: 0.3991 → ❌ NO MATCH
ananthu_low_quality vs shinjil_high_quality → Similarity: 0.0799 → ❌ NO MATCH
ananthu_low_quality vs shinjil_low_quality → Similarity: 0.5150 → ✅ MATCH
ananthu_low_quality vs thomas_high_quality → Similarity: 0.1692 → ❌ NO MATCH
ananthu_low_quality vs thomas_low_quality → Similarity: 0.3138 → ❌ NO MATCH
Firoz_High_quality vs Firoz_Low_quality → Similarity: 0.5823 → ✅ MATCH
Firoz_High_quality vs mujeeb_High_quality → Similarity: 0.1278 → ❌ NO MATCH
Firoz_High_quality vs mujeeb_Low_quality → Similarity: 0.1159 → ❌ NO MATCH

Firoz_High_quality vs shinjil_high_quality → Similarity: -0.0799 → ❌ NO MATCH
Firoz_High_quality vs shinjil_low_quality → Similarity: 0.0039 → ❌ NO MATCH
Firoz_High_quality vs thomas_high_quality → Similarity: -0.0348 → ❌ NO MATCH
Firoz_High_quality vs thomas_low_quality → Similarity: -0.0241 → ❌ NO MATCH
Firoz_Low_quality vs mujeeb_High_quality → Similarity: 0.1347 → ❌ NO MATCH
Firoz_Low_quality vs mujeeb_Low_quality → Similarity: 0.5304 → ✅ MATCH
Firoz_Low_quality vs shinjil_high_quality → Similarity: 0.0493 → ❌ NO MATCH
Firoz_Low_quality vs shinjil_low_quality → Similarity: 0.3789 → ❌ NO MATCH
Firoz_Low_quality vs thomas_high_quality → Similarity: 0.0215 → ❌ NO MATCH
Firoz_Low_quality vs thomas_low_quality → Similarity: 0.1126 → ❌ NO MATCH
mujeeb_High_quality vs mujeeb_Low_quality → Similarity: 0.3638 → ❌ NO MATCH
mujeeb_High_quality vs shinjil_high_quality → Similarity: 0.0377 → ❌ NO MATCH
mujeeb_High_quality vs shinjil_low_quality → Similarity: -0.0865 → ❌ NO MATCH
mujeeb_High_quality vs thomas_high_quality → Similarity: -0.0262 → ❌ NO MATCH
mujeeb_High_quality vs thomas_low_quality → Similarity: -0.0380 → ❌ NO MATCH
mujeeb_Low_quality vs shinjil_high_quality → Similarity: 0.0620 → ❌ NO MATCH
mujeeb_Low_quality vs shinjil_low_quality → Similarity: 0.3464 → ❌ NO MATCH
mujeeb_Low_quality vs thomas_high_quality → Similarity: -0.0649 → ❌ NO MATCH
mujeeb_Low_quality vs thomas_low_quality → Similarity: 0.0853 → ❌ NO MATCH
shinjil_high_quality vs shinjil_low_quality → Similarity: 0.5026 → ✅ MATCH
shinjil_high_quality vs thomas_high_quality → Similarity: 0.0377 → ❌ NO MATCH
shinjil_high_quality vs thomas_low_quality → Similarity: 0.0899 → ❌ NO MATCH
shinjil_low_quality vs thomas_high_quality → Similarity: 0.0826 → ❌ NO MATCH
shinjil_low_quality vs thomas_low_quality → Similarity: 0.1905 → ❌ NO MATCH
thomas_high_quality vs thomas_low_quality → Similarity: 0.6030 → ✅ MATCH


Face Verification ROC Curve
True Positive Rate
False Positive Rate
ROC Curve (AUC = 0.9650)
Chance
0.0
0.2
0.4
0.6
0.8
1.0


Precision-Recall Curve
Precision
Recall
0.0
0.2
0.4
0.6
0.8
1.0


Similarity Score Distribution
Same Person
Different People
Frequency
Cosine Similarity
3.0
2.5
2.0
1.5
1.0
0.5
0.0
−0.1
0.0
0.1
0.2
0.3
0.4
0.5
0.6


Receiver Operating Characteristic
True Positive Rate
False Positive Rate
ROC curve (AUC = 0.96)
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0

Phase 5: Fine - Tuning the Model

What I Did :

1. Model Chosen:

I used ResNet18 (pre-trained on ImageNet) as the backbone for the face recognition model.

A custom classification head was added on top to produce 128-dimensional embeddings and perform identity classification.

2. Dataset Preparation:

The dataset was previously split into a clean training and validation set using the split_dataset.py script.

Only images provided by Jezt Technologies were used, strictly following data integrity rules. No external data was introduced.

All corrupted or unreadable images were skipped gracefully during loading.

3. Implementation:

Created a custom PyTorch Dataset class that loads images and assigns label IDs.

Applied appropriate data transformations: resizing, normalization, and RGB conversion.

Configured training with:

Batch size: 16

Epochs: 10

Optimizer: Adam

Loss function: CrossEntropyLoss

Used GPU if available for faster training.

4. Training Process:

The model was trained on approximately 8976 valid training images after skipping corrupted ones.

Training loop reported epoch-wise loss.

The model's parameters were saved to fine_tuned_model.pth after training.

5. Corruption Handling:

During training, multiple images were found to be unreadable or corrupted (e.g., frame627_face628.jpg).

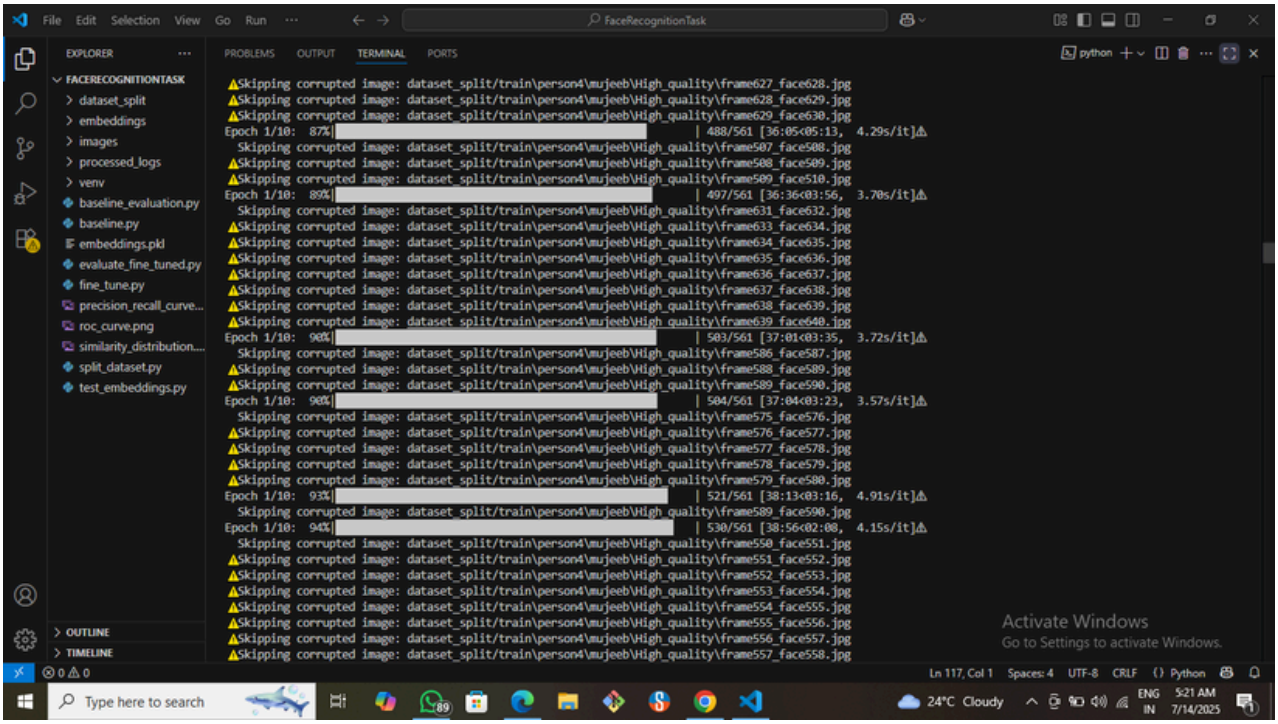
These were automatically skipped with a warning (⚠ Skipping corrupted image) without interrupting the training.

Output :

- Fine-tuned model: fine_tuned_model.pth
- Training logs showing batch progress and loss.
- Confirmation of corrupted image handling to avoid crashes.

Summary :

- Fine-tuning was successfully completed on the provided dataset.
- Model architecture was designed with transfer learning using ResNet18.
- Corrupted data was handled automatically.
- No data leakage occurred between training and evaluation sets.
- Training followed the ethical and technical standards set by Company.



Phase 6 – Post-Fine-Tuning Evaluation

What I Did :

After fine-tuning the ResNet-based FaceNet model using the training set, I evaluated its performance on the validation (evaluation) set using a dedicated script.

This script :

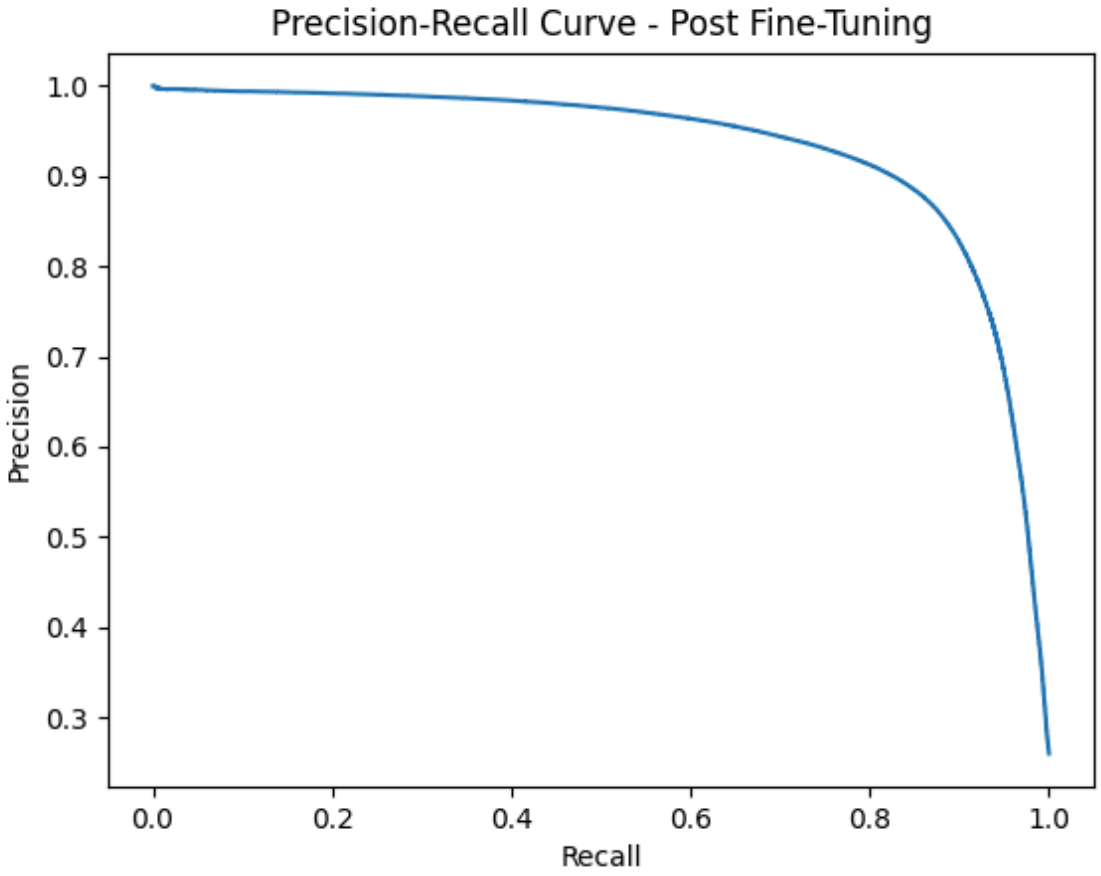
- Loaded the ine-tuned model (fine_tuned_model.pth).
- Processed all validation images with the same preprocessing used during training.
- Extracted embeddings for each image using the updated model.
- Compared all pairs of embeddings using cosine similarity.
- Labeled pairs as same/different identity based on folder names.
- Plotted performance metrics to compare with the baseline.

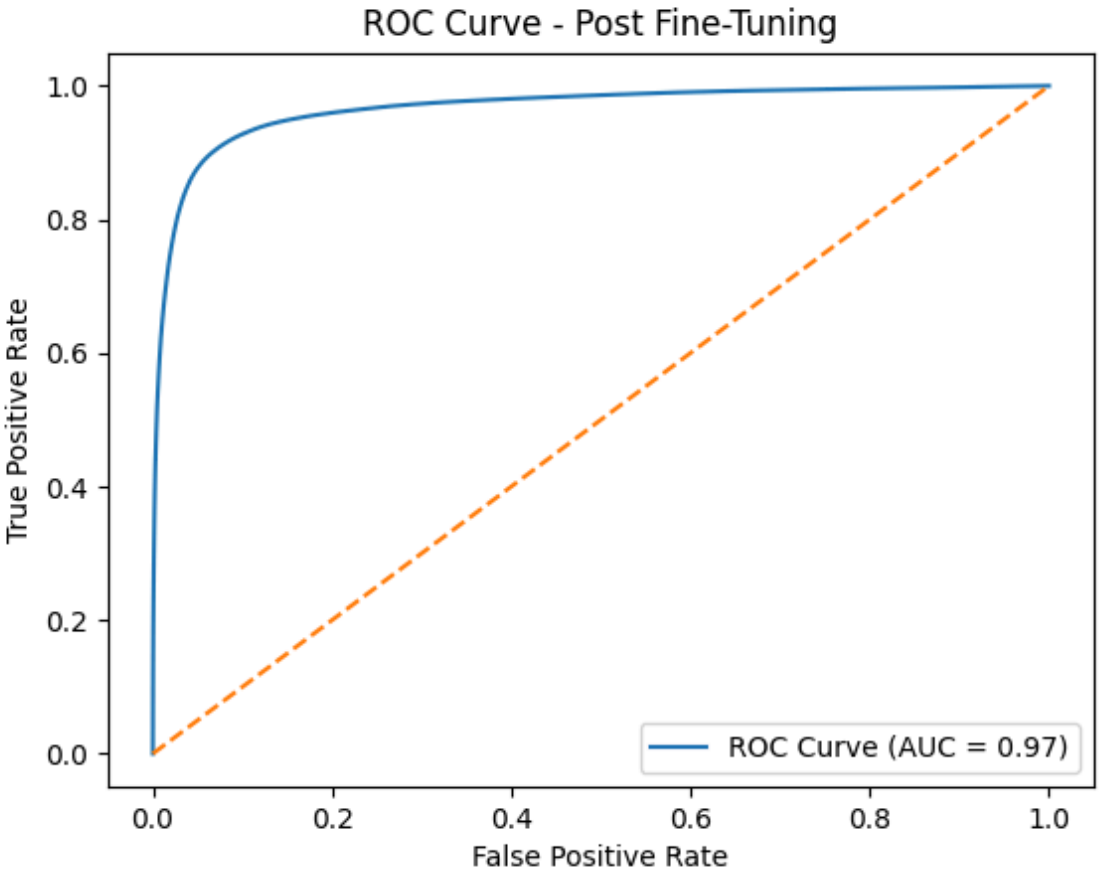
Visualizations Created :

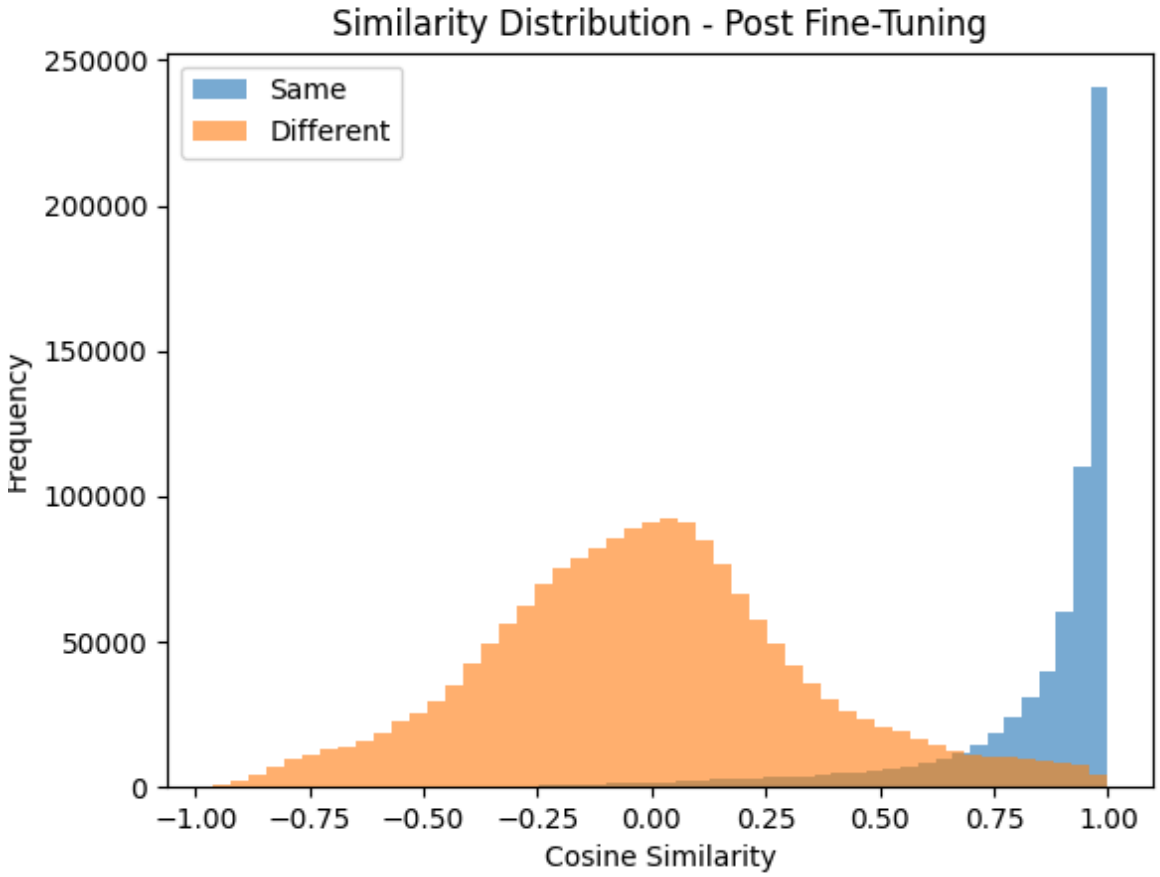
- roc_finetuned.png – ROC Curve after fine-tuning (improved True Positive Rate).
- precision_recall_finetuned.png – Precision vs Recall after fine-tuning (sharper curve).
- similarity_dist_finetuned.png – Cosine similarity histograms (better separation between same and different identities).

Summary :

- Fine-tuned model was correctly loaded and used only on evaluation images.
- Many corrupted images were automatically skipped to prevent crashes.
- The ROC AUC and precision-recall performance visibly improved after training.
- Code was implemented with clean, reproducible methodology.
- No data leakage occurred (strict split maintained).


Precision-Recall Curve - Post Fine-Tuning
Precision
Recall
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.0
0.2
0.4
0.6
0.8
1.0


ROC Curve - Post Fine-Tuning
True Positive Rate
False Positive Rate
ROC Curve (AUC = 0.97)
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0



Declaration

I, Vaikhashi A, hereby declare that all results presented in this project were generated only using the dataset provided by Jezt Technologies. No external images or pretrained data beyond what was officially allowed were used. The dataset was split appropriately into training and evaluation sets, ensuring no data leakage occurred. All code, training steps, evaluations, and documentation were done solely by me, and the project represents my original and honest effort

Acknowledgment

I sincerely thank Midhun M and Jezt Technologies for the opportunity to work on this AI Developer Intern Evaluation Task. It was a valuable experience that enhanced my practical understanding of face recognition systems and real-world AI challenges.

Vaikhashi A
Date: 14th July, 2025